

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Ano Letivo de 2025/2026

Fase 4: Normais e Coordenadas de Textura

Carlos Araújo
A106845

João Teixeira
A106836

Simão Mendes
A106928

Vitor Senra
A106921

May 08, 2026

CG

Índice

1. Introdução 1

2. Iluminação 2
2.1. Cálculo Matemático das Normais (Generator) 2
2.2. Processamento e Aplicação de Luzes (Engine) 5
2.3. Componentes dos Materiais 6

3. Texturas 7
3.1. Estrutura de Dados e Formato de Ficheiro 7
3.2. Mapeamento UV por Primitiva 7
3.2.1. Plano 8
3.2.2. Caixa 8
3.2.3. Esfera 9
3.2.4. Cone 10
3.2.5. Torus 11
3.2.6. Cometa 12
3.2.7. Superfícies Bézier (Patch) 12
3.3. Pipeline de Texturas no Motor de Renderização 13
3.3.1. Integração da Biblioteca DevIL 14
3.3.2. Renderização com VBOs e Estados OpenGL 14
3.3.3. Configuração XML 14

4. Resultados 15
4.1. Resultados e Validação 15
4.2. Descrição da Cena 18
4.3. Hierarquia e Organização da Cena XML 19
4.4. Texturas Aplicadas na Cena 20
4.5. Configuração de Iluminação e Materiais 21
4.6. Resultado Visual Final 23

5. Conclusão 25

6. Referências Bibliográficas 26

1. Introdução

O presente relatório detalha o desenvolvimento da Fase 4 do nosso Motor 3D, marcando o culminar de um projeto iterativo de Computação Gráfica. Desde a Fase 1, a arquitetura do motor tem sofrido uma evolução significativa, adaptando-se a requisitos cada vez mais complexos e ambiciosos.

Inicialmente, o nosso foco residia na construção de primitivas geométricas simples e na sua visualização básica. Na Fase 2, introduzimos as transformações geométricas e a estruturação da cena através de um grafo de cena hierárquico (Scene Graph), permitindo a construção de modelos mais complexos. A Fase 3 representou um salto importante com a implementação de *Vertex Buffer Objects* (VBOs) para otimização do desenho e a adoção do formato XML para a definição das cenas.

Agora, nesta Fase 4, que representa a etapa final do projeto, atingimos a maturidade do sistema. O motor não só desenha geometria otimizada como também simula o comportamento da luz de forma realista, utilizando diferentes tipos de fontes luminosas (Pontual, Direcional e *Spotlight*). Para que a iluminação seja calculada corretamente pelo OpenGL, todos os modelos exportados pelo nosso *Generator* incluem agora as respetivas **Normais**. Adicionalmente, implementámos com sucesso o mapeamento de texturas, calculando e utilizando as **Coordenadas de Textura** (UVs) para conferir cor e um elevado nível de detalhe às superfícies dos nossos modelos.

A arquitetura geral do nosso sistema consolidou-se em dois pilares principais: o **Generator** (responsável pela modelação matemática e geração dos vértices, normais e coordenadas de textura) e o **Engine** (encarregue do processamento das estruturas de dados e da renderização gráfica em tempo real). A comunicação e parametrização entre estes dois módulos independentes é assegurada através de ficheiros de configuração XML. O diagrama abaixo ilustra este fluxo de trabalho.

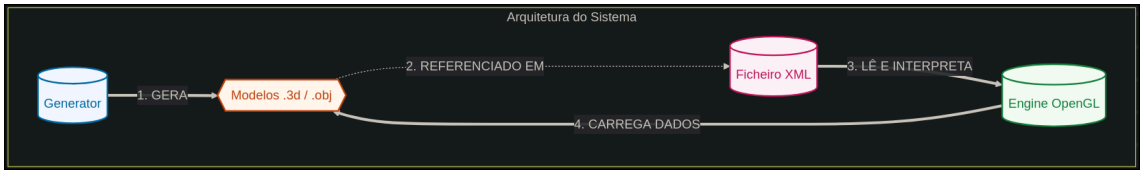


Figura 1: Arquitetura geral do sistema demonstrando o fluxo entre o Generator, XML e o Engine.

2. Iluminação

Nesta quarta fase do projeto, o foco principal incidiu sobre a introdução de realismo visual na nossa cena 3D através da implementação de iluminação e do cálculo rigoroso de materiais e propriedades de reflexão. Para que a luz atue corretamente sobre os modelos, tornou-se imperativo gerar as normais no nosso *Generator* e configurar devidamente as fontes de luz e propriedades dos materiais no *Engine*.

2.1. Cálculo Matemático das Normais (Generator)

Para que o OpenGL consiga calcular a iluminação de forma correta utilizando o modelo de reflexão de Phong, é necessário fornecer um vetor normal para cada vértice gerado. A normal é um vetor perpendicular à superfície no ponto em questão e dita como a luz interage com a mesma.

A matemática por trás do cálculo varia consoante o tipo de primitiva:

1. **Primitivas Básicas (Planos e Caixas):**
   Dado que as superfícies são perfeitamente planas e alinhadas com os eixos, a normal de qualquer vértice numa face é perpendicular a essa mesma face. Por exemplo, para um plano assente no eixo XZ (virado para cima), a normal em todos os vértices é trivialmente o vetor unitário do eixo Y:

$$\vec{N} = (0, 1, 0)$$

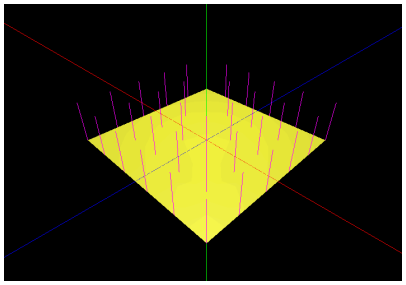


Figura 3: Teste de Normais Plane

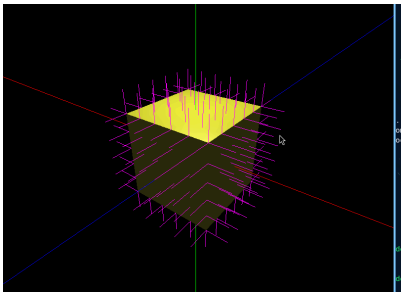


Figura 4: Teste de Normais Box

2. **Esfera:**
   Para uma esfera centrada na origem, a matemática simplifica-se significativamente. A direção perpendicular à superfície num determinado vértice $V(x, y, z)$ aponta exatamente do centro $(0, 0, 0)$ para o próprio vértice. Assim, a normal é a normalização do próprio vetor posição do vértice:

$$\vec{N} = \frac{x, y, z}{\sqrt{x^2 + y^2 + z^2}}$$

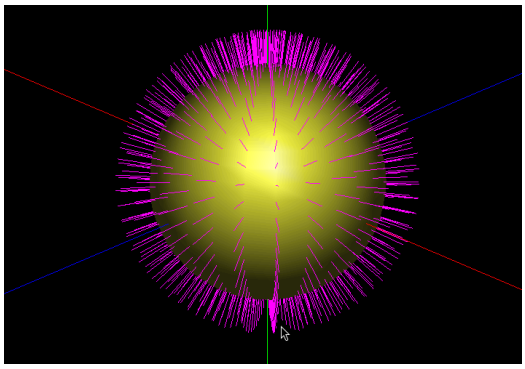


Figura 5: Teste de Normais Sphere

3. **Superfícies de Bézier (Patches):**
   Para as superfícies paramétricas, a normal é calculada com recurso ao produto vetorial entre as derivadas parciais da superfície em relação aos parâmetros u e v. Sendo a superfície avaliada numa função $P(u, v)$, calculamos os vetores tangentes:

   $$\vec{T}_u = \frac{\partial P}{\partial u} \quad \text{e} \quad \vec{T}_v = \frac{\partial P}{\partial v}$$

   A normal da superfície nesse vértice é obtida pelo produto vetorial destes dois vetores, garantindo a ortogonalidade, e posteriormente normalizada:

   $$\vec{N} = \frac{\vec{T}_u \times \vec{T}_v}{\left\| \vec{T}_u \times \vec{T}_v \right\|}$$

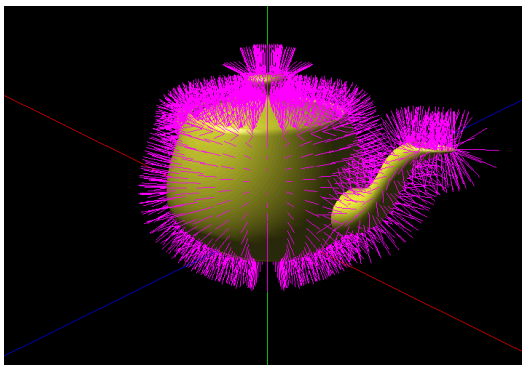


Figura 6: Teste de Normais Teapot

4. **Cone:**
   O cálculo das normais de um cone divide-se em duas partes. Para a base, a superfície é perfeitamente plana, pelo que a normal aponta estritamente no sentido negativo do eixo vertical (assumindo o alinhamento no eixo Y):

   $$\vec{N} = (0, -1, 0)$$

   Para a superfície lateral, a normal tem de ser perpendicular à inclinação do cone. Sabendo o raio da base R, a altura H e a posição de um vértice $V(x, y, z)$, o vetor aponta radialmente para fora, mas com uma inclinação ascendente (tilt). A normal não normalizada pode ser obtida por:

$$\vec{N}_{\text{lateral}} = \left(x, \frac{R}{H}\sqrt{x^2 + z^2}, z \right)$$

Sendo posteriormente normalizada para a sua aplicação final no motor.

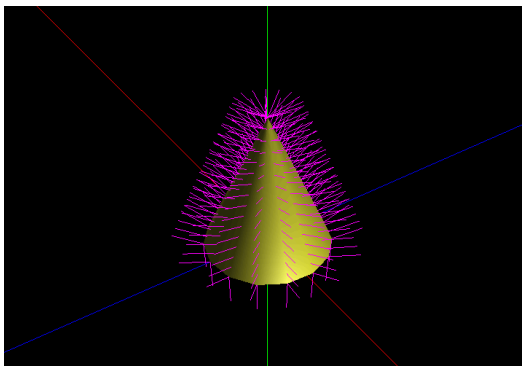


Figura 7: Teste de Normais Cone

5. **Torus:**
   O Torus é uma figura geométrica gerada pela revolução de um círculo menor (tubo de raio r) ao longo do trajeto de um círculo maior (anel de raio R). A normal num vértice $V(x, y, z)$ aponta exatamente do centro da secção transversal do tubo para esse mesmo vértice. Sendo θ o ângulo da fatia principal (o anel), o ponto central do tubo nesse instante é $C = (R\cos(\theta), 0, R\sin(\theta))$. A normal exata é simplesmente o vetor que vai de C até ao vértice V, dividida pelo raio do tubo para garantir a normalização:

$$\vec{N} = \frac{V - C}{r} = \frac{x - R\cos(\theta), y, z - R\sin(\theta)}{r}$$

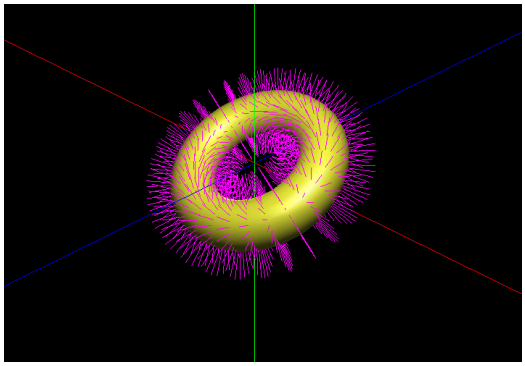


Figura 8: Teste de Normais Torus

6. **Cometa (Superfície Complexa/Deformada):**
   Sendo o Cometa uma primitiva geométrica extra e de formato irregular (frequentemente gerado através da deformação matemática de uma esfera ou por funções paramétricas assimétricas), o cálculo analítico direto torna-se complexo. Nestes casos, o nosso *Generator* garante normais precisas recorrendo novamente ao produto vetorial dos vetores tangentes. Caso a deformação impossibilite a derivação analítica exata, o método de diferenças finitas pode ser empregue para

estimar os vetores tangentes $\vec{T}_u$ e $\vec{T}_v$ com base nos pontos vizinhos na grelha paramétrica:

$$\vec{T}_u \approx \frac{P(u+\Delta u, v) - P(u-\Delta u, v)}{2\Delta u}$$

A normal é então obtida, tal como nas superfícies de Bézier, pelo produto vetorial e respetiva normalização:

$$\vec{N} = \frac{\vec{T}_u \times \vec{T}_v}{\left\|\vec{T}_u \times \vec{T}_v\right\|}$$

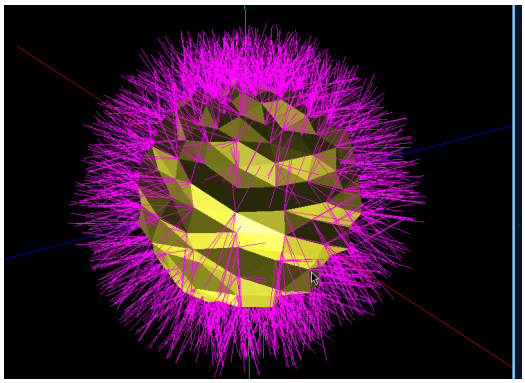


Figura 9: Teste de Normais Cometa

2.2. Processamento e Aplicação de Luzes (Engine)

No nosso *Engine*, a leitura e configuração das luzes ocorre aquando do parse do ficheiro XML, especificamente dentro da tag `<lights>`. O OpenGL (em versão *fixed-function pipeline*) permite o uso de até 8 luzes simultâneas (`GL_LIGHT0` a `GL_LIGHT7`). O motor lê as luzes especificadas e, conforme o atributo `type`, configura os parâmetros adequados usando a função `glLightfv`:

- **Point Light (Luz Pontual):** Uma luz que emite radiação em todas as direções a partir de uma posição específica no espaço. O parser extrai as componentes `posX`, `posY` e `posZ` do XML. No OpenGL, passamos esta posição com a componente homogénea $w = 1.0$.
- **Directional Light (Luz Direcional):** Simula uma fonte de luz infinitamente distante (como o Sol visto da terra), emitindo raios paralelos num vetor específico. O XML fornece as componentes `dirX`, `dirY` e `dirZ`. No OpenGL, utilizamos este vetor como o parâmetro `GL_POSITION`, mas atribuindo a componente homogénea $w = 0.0$, o que sinaliza tratar-se de uma direção.
- **Spotlight (Luz de Foco):** Este tipo de luz necessita de uma posição espacial e de uma direção de foco, formando um cone de luz. O motor lê os parâmetros posicionais (`posX`, `posY`, `posZ`) e direcionais (`dirX`, `dirY`, `dirZ`), juntamente com o ângulo de abertura `cutoff`. Em OpenGL, para além da posição ($w = 1.0$), utilizamos os parâmetros `GL_SPOT_DIRECTION` e `GL_SPOT_CUTOFF`.

2.3. Componentes dos Materiais

A forma como um modelo reage às luzes configuradas é determinada pelo seu material. O motor foi expandido para processar a tag `<color>` dentro de um `<model>`, aplicando os valores através da função `glMaterialfv`.

O XML permite definir cinco componentes cruciais de iluminação do modelo de Phong:

- **Diffuse:** A cor base do objeto quando atingido de forma direta por luz. Determina a cor primária que os nossos olhos percecionam.
- **Ambient:** A cor do objeto em áreas onde não há luz direta (zonas de sombra), simulando a luz que "salta" no ambiente.
- **Specular:** A componente que simula o brilho do reflexo direto da luz (o "ponto branco" brilhante num material plástico ou metálico).
- **Emissive:** Permite que o objeto emita a sua própria cor, ignorando as fontes de luz da cena (não ilumina outros objetos, mas brilha no escuro).
- **Shininess:** Um valor escalar (tipicamente entre 0 e 128) lido a partir do atributo `value` que dita o raio/foco do brilho especular. Valores altos resultam num reflexo mais pequeno e focado (mais metálico).

Para precaver falhas ou falta de formatação no XML, o *Engine* implementa valores de salvaguarda (default) caso a cor não seja definida. Conforme a especificação, os padrões assumidos são componentes *diffuse* de `(200, 200, 200)`, *ambient* de `(50, 50, 50)`, *specular* e *emissive* nulos `(0, 0, 0)`, e um brilho (*shininess*) a 0.

Para validar a correta implementação do modelo de reflexão de luz, configurou-se uma cena com dois objetos idênticos sujeitos à mesma iluminação, mas com propriedades de material (lidas via XML) completamente diferentes. Como ilustrado na figura abaixo, a esfera vermelha simula um material mate, como argila ou borracha, utilizando apenas componentes ambiente e difusas. Já a esfera azul evidencia o funcionamento da componente especular (specular highlight), criando o efeito de um material brilhante e plástico devido ao elevado valor de shininess atribuído. O contraste entre os dois modelos confirma que o motor distingue e aplica corretamente os diferentes tipos de superfícies.

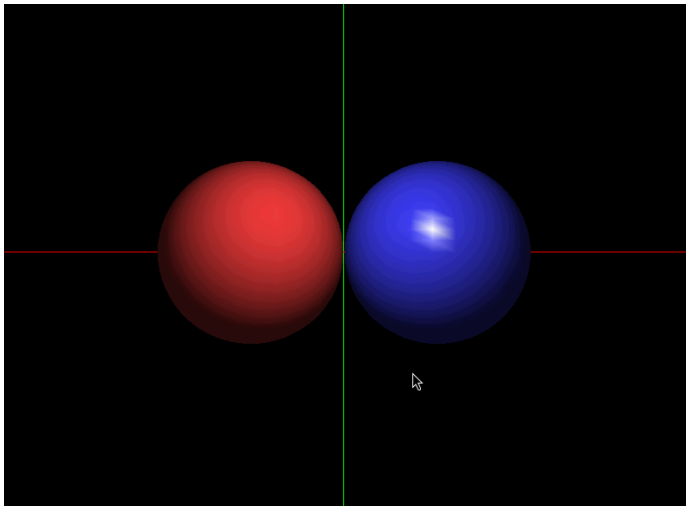


Figura 10: Teste de Componentes de Materiais

3. Texturas

O mapeamento de texturas é uma das técnicas fundamentais para aumentar o realismo visual em cenas de computação gráfica. Em vez de representar a aparência de uma superfície apenas através de propriedades de cor e iluminação, o mapeamento UV permite "pintar" uma imagem 2D (a textura) sobre a geometria 3D. Cada vértice da malha carrega, para além das coordenadas espaciais (x, y, z) e do vetor normal (n_x, n_y, n_z), um par de coordenadas de textura (u, v), normalmente definidas no intervalo $[0, 1]$.

No âmbito deste projeto, o **Generator** é responsável por calcular as coordenadas UV para cada primitiva geométrica no momento da geração do modelo, enquanto o **Engine** carrega as imagens de textura e aplica-as durante a renderização via OpenGL. Este capítulo descreve a lógica matemática do mapeamento UV implementado para cada primitiva (plano, caixa, esfera, cone, torus, cometa e superfícies Bézier), o formato de ficheiro utilizado para persistir esses dados, e o pipeline de carregamento e aplicação de texturas no motor gráfico.

3.1. Estrutura de Dados e Formato de Ficheiro

Toda a geometria gerada pelo **Generator** partilha uma estrutura de vértice unificada, onde cada vértice agrega oito valores escalares: as três coordenadas espaciais, as três componentes do vetor normal unitário e o par de coordenadas de textura (tu, tv).

A persistência dos modelos é feita num formato inspirado no Wavefront OBJ, introduzido na Fase 3 do projeto. Cada ficheiro `.3d` contém duas secções distintas:

- **Vértices**: uma linha por vértice único, identificada pelo prefixo `v`, contendo sequencialmente as coordenadas espaciais, o vetor normal e as coordenadas de textura.
- **Faces**: uma linha por triângulo, identificada pelo prefixo `f`, com três índices baseados em 1 que referenciam os vértices previamente definidos.

A deduplicação de vértices é realizada através de uma estrutura de dados ordenada que compara, com uma tolerância de $10^{\{-5\}}$, a posição espacial, o vetor normal e as coordenadas UV. A inclusão das coordenadas de textura na chave de comparação é deliberada e essencial: vértices com a mesma posição espacial mas coordenadas UV distintas devem ser preservados como entradas separadas. Este comportamento é particularmente crítico nos polos da esfera, onde a geometria degenera num único ponto espacial mas requer múltiplas projeções UV para evitar **artefacts** de costura na textura.

3.2. Mapeamento UV por Primitiva

A atribuição das coordenadas (u, v) varia consoante a topologia de cada primitiva. A convenção adotada posiciona a origem do espaço UV no canto inferior esquerdo da

textura $(0, 0)$, com $(1, 1)$ no canto superior direito, alinhando-se com a convenção nativa do OpenGL e da biblioteca DevIL.

3.2.1. Plano

O plano é definido no espaço XZ, centrado na origem, com dimensão `length` e subdividido numa grelha de `divisions` × `divisions`. Para cada célula da grelha, as coordenadas UV são calculadas linearmente em função dos índices de iteração i (eixo X) e j (eixo Z):

$$u = \frac{i}{\text{divisions}}$$

$$v = 1 - \frac{j}{\text{divisions}}$$

A inversão do eixo v é necessária porque o gerador percorre a grelha de $j = 0$ (frente, $Z = -\text{half}$) até $j = \text{divisions}$ (trás, $Z = +\text{half}$), enquanto a coordenada $v = 0$ corresponde à base inferior da textura. Sem esta inversão, a textura apareceria invertida verticalmente no ecrã.

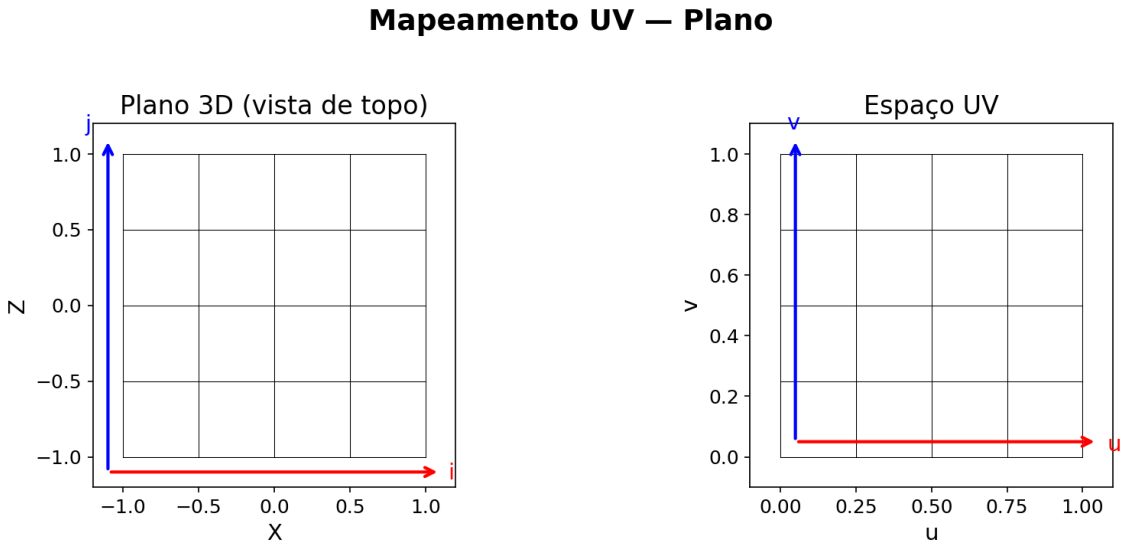


Figura 11: Mapeamento UV do plano — grelha 3D e correspondência no espaço UV

3.2.2. Caixa

A caixa é composta por seis faces independentes, cada uma subdividida numa grelha `grid` × `grid`. O mapeamento UV de cada face é uma projecção ortogonal direta da grelha para o quadrado unitário $[0, 1]^2$:

$$u = \frac{i}{\text{grid}}$$

$$v = \frac{j}{\text{grid}}$$

Não há inversão do eixo v na caixa, porque cada face é gerada de forma independente e a orientação dos triângulos é coerente com a disposição da textura. O diagrama seguinte ilustra o **unwrapping** conceptual das seis faces no plano UV (layout em cruz).

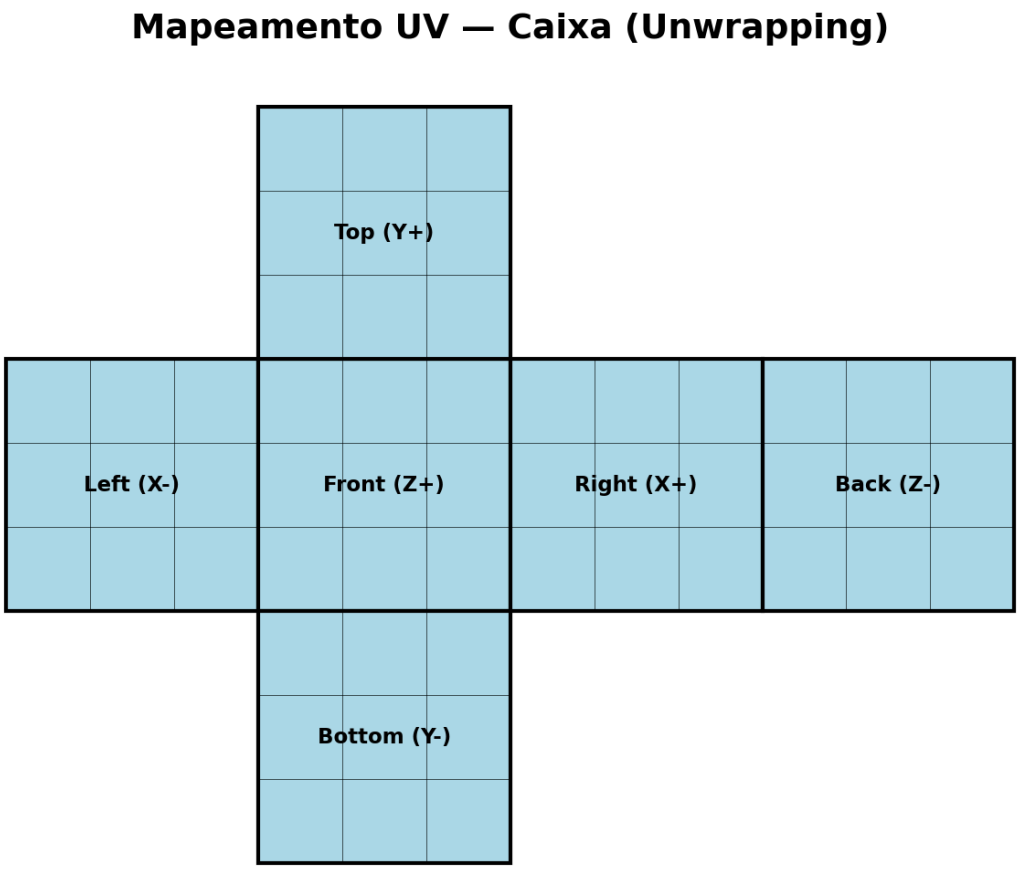


Figura 12: Unwrapping das 6 faces da caixa no plano UV

3.2.3. Esfera

A esfera é parametrizada por coordenadas esféricas, onde $\alpha \in [0, 2\pi]$ (ângulo azimutal, dividido em `slices`) e $\beta \in [0, \pi]$ (ângulo polar, dividido em `stacks`). As coordenadas espaciais são dadas por:

$$x = R \cdot \sin(\beta) \cdot \sin(\alpha)$$

$$y = R \cdot \cos(\beta)$$

$$z = R \cdot \sin(\beta) \cdot \cos(\alpha)$$

O vetor normal em cada ponto é simplesmente o vetor unitário radial $\boldsymbol{n} = \left(\frac{x}{R}, \frac{y}{R}, \frac{z}{R}\right)$. O mapeamento UV segue uma abordagem cilíndrica:

$$u = \frac{i}{\text{slices}}$$

$$v = 1 - \frac{j}{\text{stacks}}$$

Nos polos ($j = 0$, polo norte, e $j = \text{stacks} - 1$, polo sul), a geometria degenera em triângulos onde dois vértices partilham a mesma posição espacial $(0, R, 0)$ e $(0, -R, 0)$, respetivamente. Apesar da degenerescência geométrica, as coordenadas UV são mantidas distintas para evitar **artefacts** de textura nos polos. Esta preservação só é possível porque o mecanismo de deduplicação de vértices inclui as coordenadas UV na chave de comparação, mantendo entradas separadas para vértices com posições coincidentes mas projeções texturais diferentes.

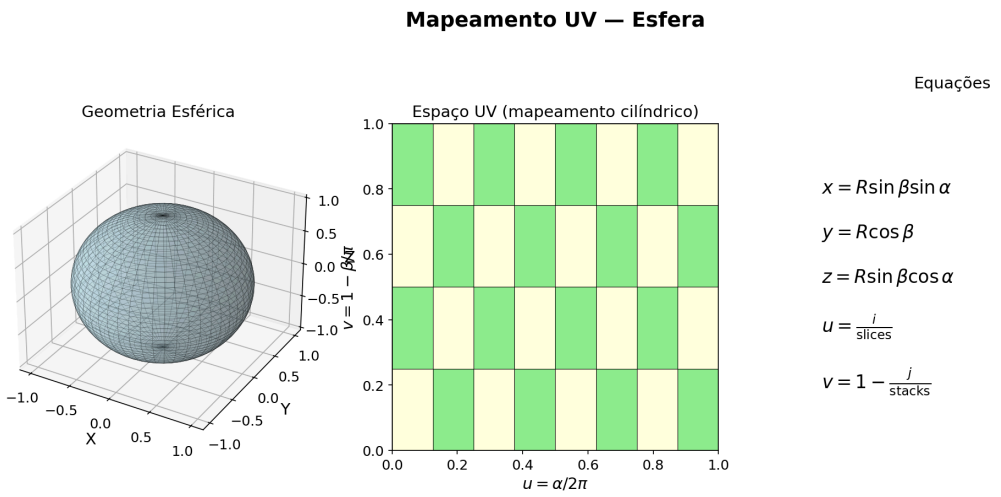


Figura 13: Mapeamento UV da esfera — geometria esférica, espaço UV e equações associadas

3.2.4. Cone

O cone divide-se em duas componentes topológicas distintas: a base circular e a superfície lateral.

Base circular. O mapeamento da base utiliza coordenadas polares centradas no ponto $(0.5, 0.5)$ do espaço UV:

$$u = 0.5 + 0.5 \cdot \sin(\alpha)$$

$$v = 0.5 + 0.5 \cdot \cos(\alpha)$$

onde $\alpha = i \cdot \frac{2\pi}{\text{slices}}$. O centro da base é mapeado para $(0.5, 0.5)$.

Superfície lateral. A lateral é tratada como um strip rectangular, onde u varia com o ângulo azimutal e v varia linearmente com a altura:

$$u = \frac{i}{\text{slices}}$$

$$v = \frac{j}{\text{stacks}}$$

O vetor normal da lateral é constante por **slice** e inclinado em relação ao eixo Y. Sendo h a altura, r o raio da base e $L = \sqrt{h^2 + r^2}$, as componentes da normal são:

$$n_x = \frac{h \cdot \sin(\alpha)}{L}$$

$$n_y = \frac{r}{L}$$

$$n_z = \frac{h \cdot \cos(\alpha)}{L}$$

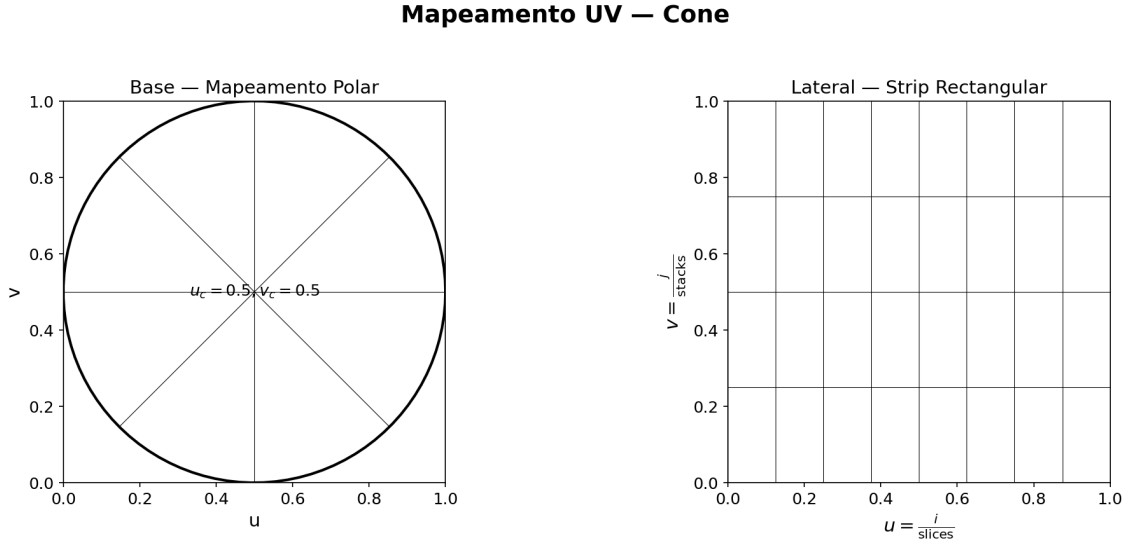


Figura 14: Mapeamento UV do cone — base circular (polar) e lateral (strip)

3.2.5. Torus

O toro é definido por dois raios — um raio exterior `outerRadius` que determina a distância do centro do tubo ao centro do toro, e um raio interior `innerRadius` que define a espessura do tubo. A superfície é parametrizada por dois ângulos periódicos: $\alpha \in [0, 2\pi]$ (dividido em `slices`, rotação em torno do eixo central do toro) e $\beta \in [0, 2\pi]$ (dividido em `stacks`, rotação em torno do centro do próprio tubo).

As coordenadas espaciais resultam da composição das duas rotações:

$$x = (\text{outerRadius} + \text{innerRadius} \cdot \cos(\beta)) \cdot \cos(\alpha)$$

$$y = \text{innerRadius} \cdot \sin(\beta)$$

$$z = (\text{outerRadius} + \text{innerRadius} \cdot \cos(\beta)) \cdot \sin(\alpha)$$

O vetor normal em cada ponto é obtido pela diferença entre a posição do vértice e o centro do tubo naquele slice, seguida de normalização, o que produz uma normal perpendicular à superfície do tubo.

O mapeamento UV aproveita a natureza periódica e envolvente do toro, estabelecendo uma correspondência direta entre os parâmetros angulares e o espaço unitário de textura:

$$u = \frac{i}{\text{slices}}$$

$$v = \frac{j}{\text{stacks}}$$

Ao contrário do plano ou da esfera, não é necessário inverter o eixo v, uma vez que a orientação da malha do toro é naturalmente coerente com a convenção de origem inferior-esquerda do espaço UV.

3.2.6. Cometa

A primitiva cometa deriva directamente da esfera, partilhando a mesma parametrização angular e o mesmo mapeamento UV cilíndrico. A diferença fundamental reside na geração da geometria: em vez de um raio constante R, cada ponto da grelha (i, j) recebe um raio deformado por um offset aleatório, controlado por um valor de **roughness** e uma semente pseudo-aleatória. Este offset é determinado uma única vez por ponto de grelha e reutilizado em todas as faces adjacentes, garantindo continuidade geométrica sem fissuras na malha.

As coordenadas espaciais seguem as mesmas equações esféricas, substituindo o raio fixo pelo raio deformado correspondente. As normais, no entanto, não podem ser calculadas pelo vetor radial simples devido à irregularidade da superfície. Em vez disso, são computadas explicitamente por produto externo dos vectores tangentes de cada triângulo, seguido de normalização unitária.

O mapeamento UV preserva a abordagem cilíndrica da esfera:

$$u = \frac{i}{\text{slices}}$$

$$v = \frac{j}{\text{stacks}}$$

Desta forma, o cometa mantém a mesma envolvência textural da esfera, mas aplica-a sobre uma superfície orgânica e irregular, útil para representar corpos celestes como asteroides ou cometas.

3.2.7. Superfícies Bézier (Patch)

As superfícies de Bézier são definidas por patches de controlo 4×4 e avaliadas através dos polinómios de Bernstein de grau 3, $B_{i(t)}$. O ponto na superfície para um par paramétrico (u, v) é:

$$S(u,v) = \sum_{i=0}^{3} \sum_{j=0}^{3} B_{i(u)} \cdot B_{j(v)} \cdot P_{ij}$$

As derivadas parciais $\partial \frac{S}{\partial} u$ e $\partial \frac{S}{\partial} v$ são calculadas usando as derivadas dos polinómios de Bernstein. O vetor normal é obtido pelo produto externo das tangentes, seguido de normalização:

$$\boldsymbol{n} = \frac{\left(\partial \frac{S}{\partial} v\right) \times \left(\partial \frac{S}{\partial} u\right)}{\left\| \left(\partial \frac{S}{\partial} v\right) \times \left(\partial \frac{S}{\partial} u\right) \right\|}$$

O mapeamento UV é direto: os próprios parâmetros paramétricos (u, v) do patch são utilizados como coordenadas de textura. Para manter a consistência com a convenção OpenGL (origem inferior-esquerda), o eixo v é invertido:

$$v_{\text{tex}} = 1 - v$$

A tesselação (resolução da amostragem) determina a densidade da malha e, consequentemente, a qualidade do mapeamento UV.

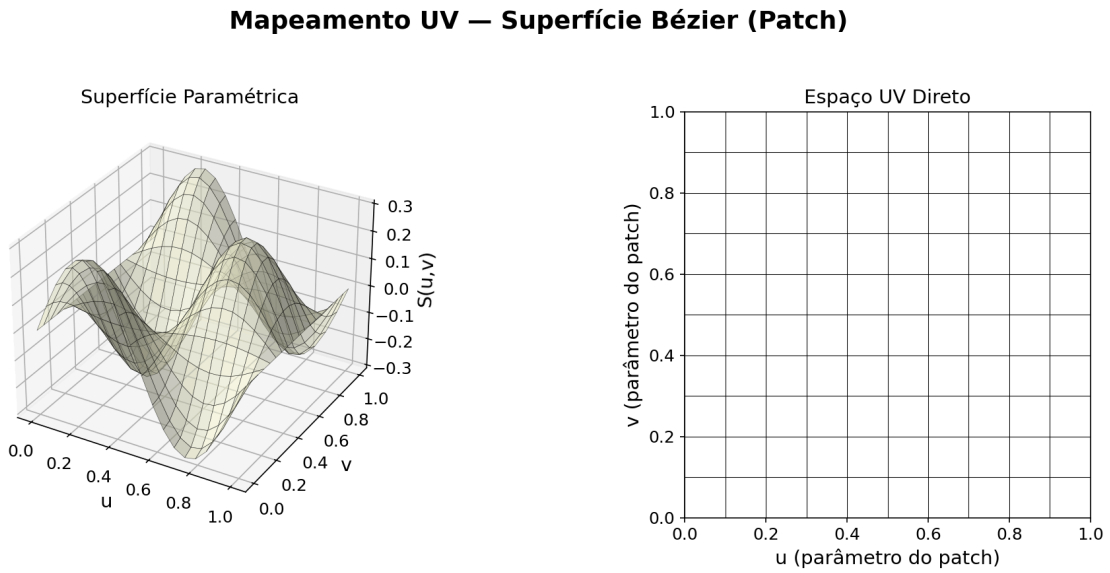


Figura 15: Mapeamento UV do patch Bézier — correspondência direta entre parâmetros paramétricos e espaço UV

3.3. Pipeline de Texturas no Motor de Renderização

O motor de renderização integra a biblioteca DevIL (Developer's Image Library) para carregamento de imagens e utiliza VBOs (Vertex Buffer Objects) do OpenGL para transferir as coordenadas UV para a GPU.

3.3.1. Integração da Biblioteca DevIL

No arranque da aplicação, a biblioteca DevIL é inicializada com a configuração de origem explícita no canto inferior esquerdo, alinhando-se com a convenção de coordenadas de textura do OpenGL.

O carregamento propriamente dito segue um pipeline sequencial. Primeiro, a imagem é carregada do disco para memória pela DevIL. Em seguida, é convertida internamente para o formato RGBA com 8 bits por canal, independentemente do formato original do ficheiro. Posteriormente, é gerado um identificador de textura OpenGL, ao qual é vinculada a imagem processada. Os parâmetros de envolvência são configurados para repetição em ambos os eixos, permitindo que a textura se repita quando as coordenadas UV excedem o intervalo unitário. A filtragem é configurada com interpolação linear para ampliação e filtragem trilinear com mipmaps para minificação. Finalmente, os dados da imagem são transferidos para a GPU e é gerada automaticamente a cadeia de mipmaps. Se o carregamento falhar, a função retorna um identificador inválido e o modelo é renderizado sem textura.

3.3.2. Renderização com VBOs e Estados OpenGL

Durante a inicialização de cada modelo, o método `prepareData` aloca quatro VBOs distintos na GPU: um para as coordenadas espaciais dos vértices, um para os índices dos triângulos, um para os vectores normais e um para as coordenadas de textura.

No momento da renderização, o pipeline avalia de forma condicional se o modelo possui uma textura válida carregada. Em caso afirmativo, activa o estado de textura bidimensional do OpenGL, vincula o identificador da textura correspondente e associa o VBO de coordenadas UV ao atributo de coordenadas de textura da GPU. Caso contrário, desactiva explicitamente o estado de textura, prevenindo que coordenadas UV residuais ou ausentes sejam incorrectamente aplicadas sobre geometria não texturizada. O desenho final é realizado através de indexação directa dos vértices quando existem índices definidos, ou em modo linear legado caso contrário.

3.3.3. Configuração XML

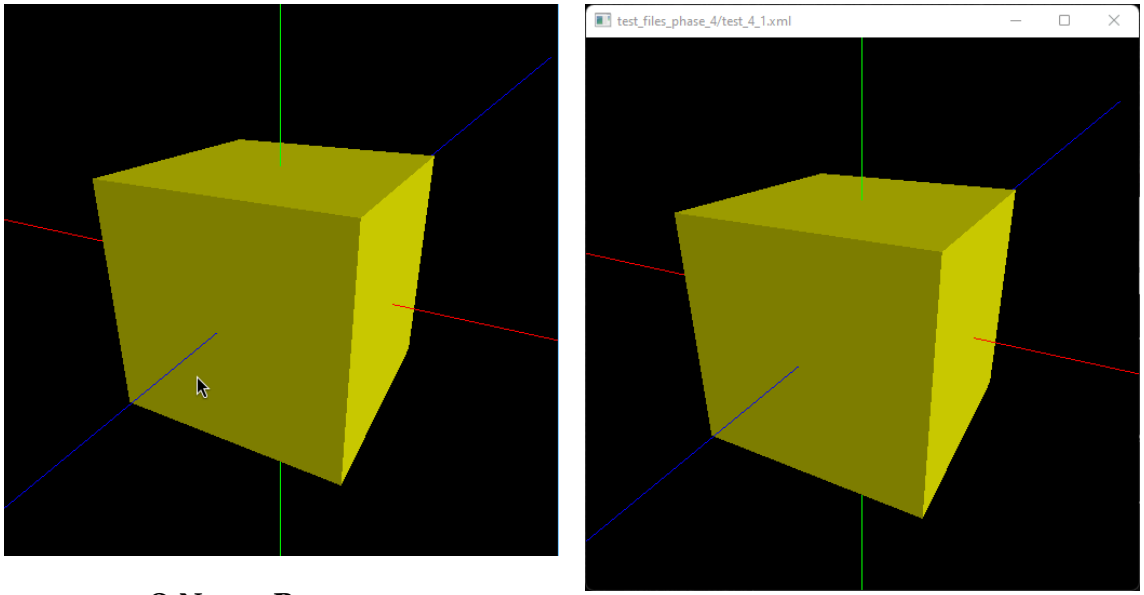
A associação entre um modelo e a sua textura é declarada no ficheiro XML da cena através de um elemento filho dentro do nó correspondente ao modelo, onde se indica o nome do ficheiro de imagem a utilizar. Durante o processamento do documento XML, o motor invoca o carregamento da textura com o caminho indicado. Se o carregamento for bem-sucedido, uma flag interna é ativada e a textura será vinculada em todas as renderizações subsequentes desse modelo. Esta abordagem permite que o mesmo ficheiro `.3d` seja reutilizado com diferentes texturas em contextos de cena distintos, sem necessidade de regenerar a geometria.

4. Resultados

4.1. Resultados e Validação

Com o intuito de validar a correta implementação do código e atestar o cumprimento dos requisitos, esta secção apresenta um conjunto de testes comparativos. A metodologia consistiu na execução do nosso `Generator` e `Engine` utilizando os cenários fornecidos, efetuando-se de seguida uma comparação visual direta entre os resultados obtidos e as imagens de referência do professor.

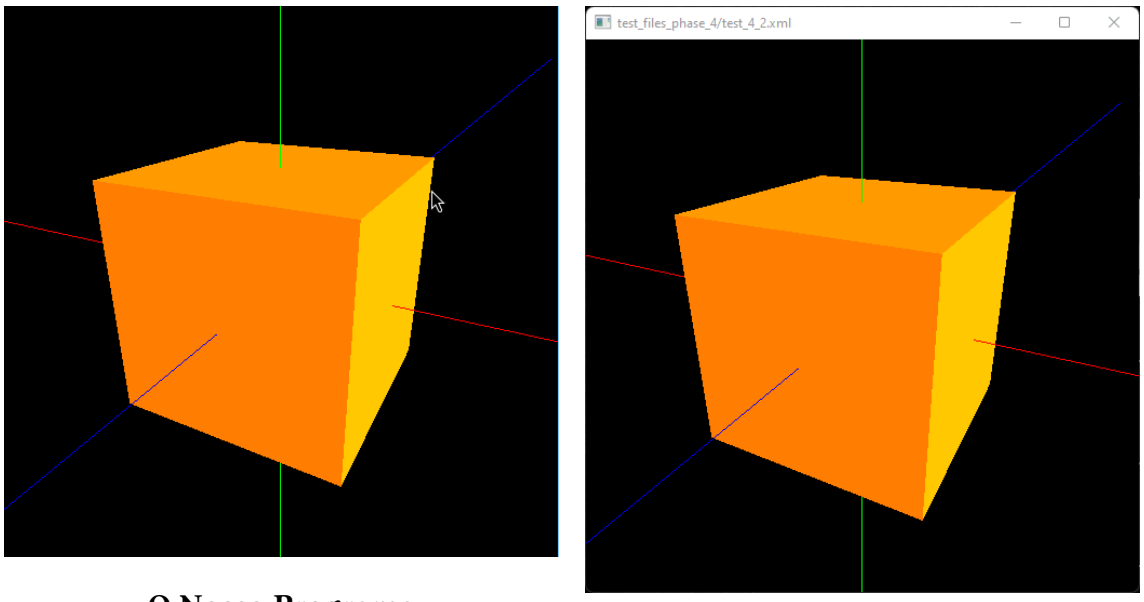
Teste 4.1



O Nosso Programa **Referência do Professor**

Figura 16: Comparação visual para o teste 4_1.

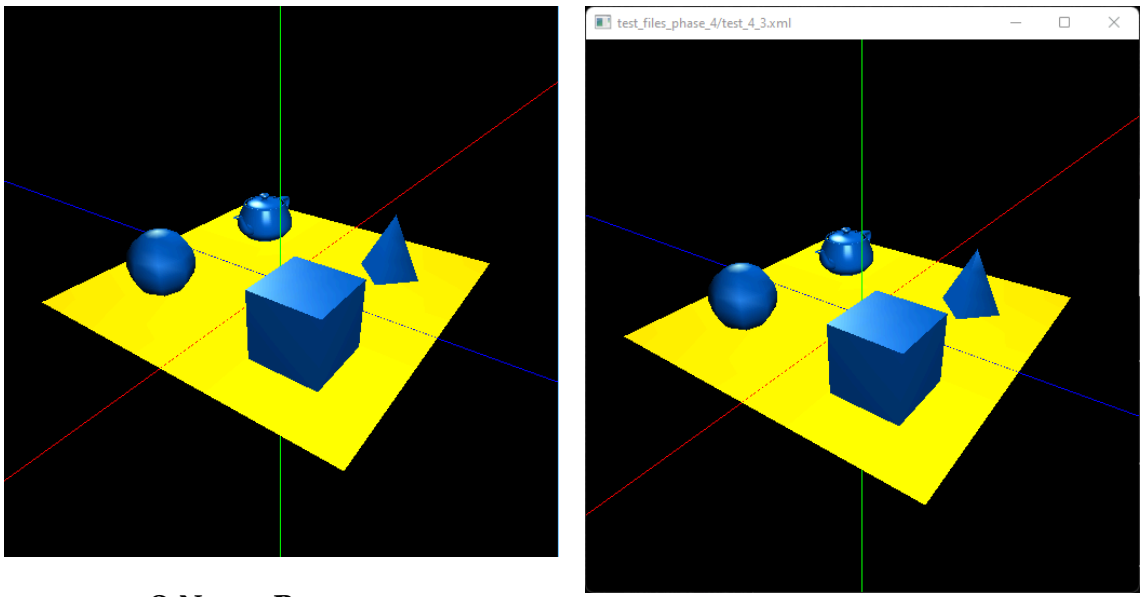
Teste 4.2



O Nosso Programa **Referência do Professor**

Figura 17: Comparação visual para o teste 4_2.

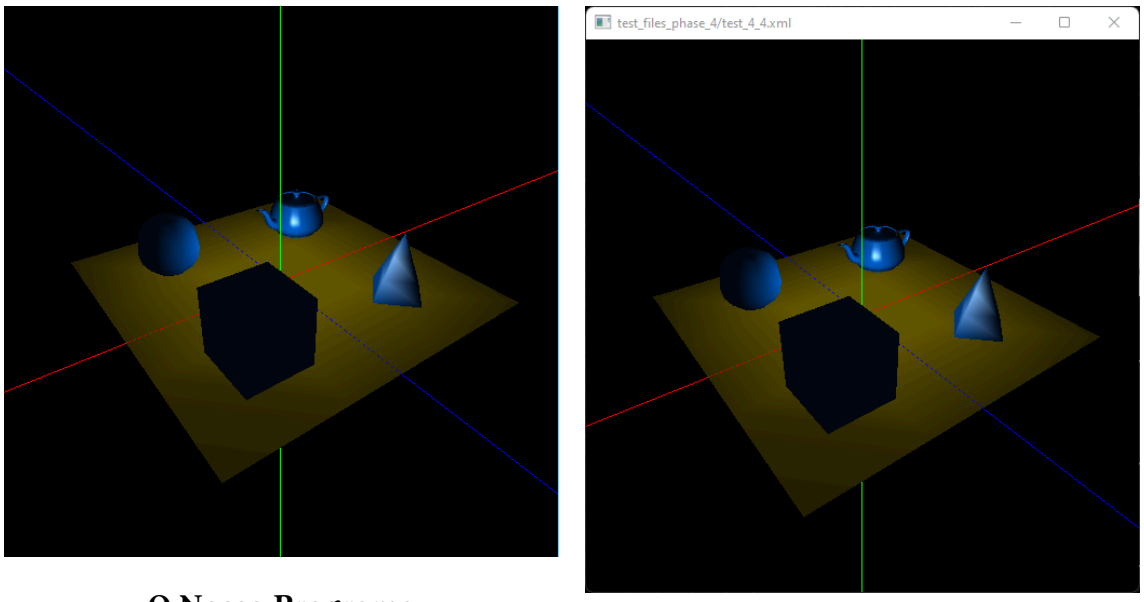
Teste 4.3



O Nosso Programa **Referência do Professor**

Figura 18: Comparação visual para o teste 4_3.

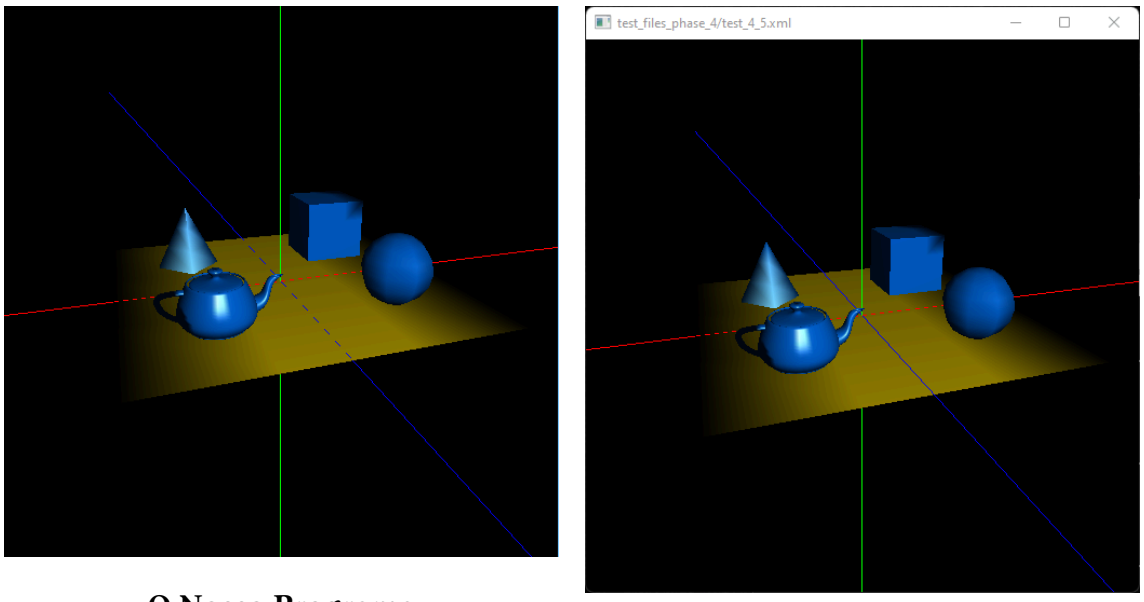
Teste 4.4



O Nosso Programa **Referência do Professor**

Figura 19: Comparação visual para o teste 4_4.

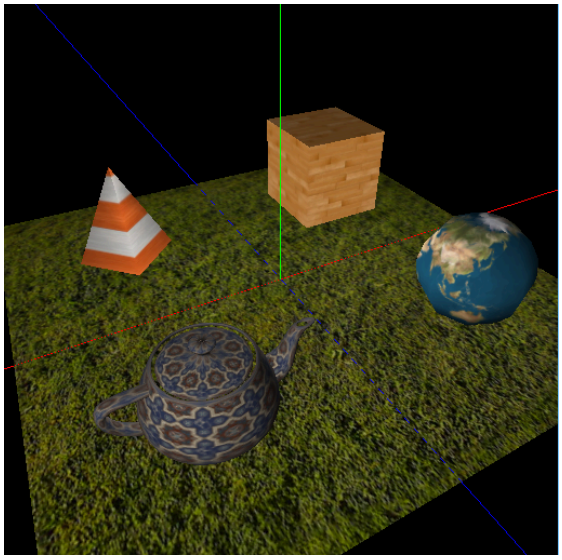
Teste 4.5



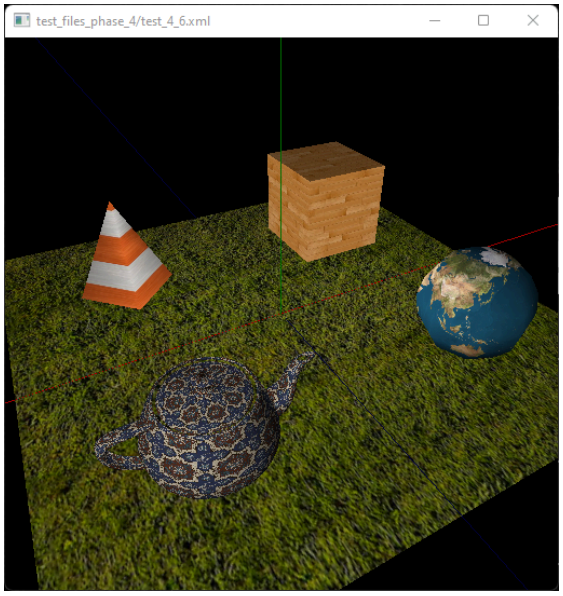
O Nosso Programa **Referência do Professor**

Figura 20: Comparação visual para o teste 4_5.

Teste 4.6



O Nosso Programa



Referência do Professor

Figura 21: Comparação visual para o teste 4_6.

4.2. Descrição da Cena

A demonstração dos resultados obtidos centra-se na construção de uma cena virtual que recria o nosso Sistema Solar. O principal objetivo desta cena é validar e exibir, de forma conjugada e harmoniosa, todas as funcionalidades implementadas no **motor** desde a Fase 1. A cena do Sistema Solar não só testa a robustez do carregamento e da **renderização** do Grafo de Cena (***Scene Graph***, detalhado no Capítulo 1), como também leva ao limite a aplicação de **normais**, coordenadas de textura e cálculos de iluminação.

A cena é composta por dezenas de instâncias de **primitivas**, incluindo o Sol, os oito planetas principais (Mercúrio, Vénus, Terra, Marte, Júpiter, Saturno, Úrano e Neptuno), satélites naturais como a Lua, Io e Europa, duas densas cinturas de asteróides contendo múltiplos fragmentos rochosos, um cometa de longo curso com órbita elíptica e uma vasta ***skybox*** invertida que projeta o fundo estrelado. Tudo isto é processado de forma hierárquica e otimizada com recurso a **VBOs** no **motor** de **renderização**.



Figura 22: Vista geral do Sistema Solar renderizado no motor, demonstrando o escalonamento das órbitas e a distribuição dos corpos celestes no ecrã.

4.3. Hierarquia e Organização da Cena XML

Conforme estabelecido no Capítulo 1, a comunicação entre o ***Generator*** e o ***Engine*** é suportada por **ficheiros `XML`** estruturados de forma hierárquica. Na cena em análise, o ficheiro `sistemasolar.xml` explora exaustivamente a propriedade de aninhamento da ***tag*** `<group>`.

Cada planeta possui o seu próprio conjunto de transformações. Para garantir que as luas orbitam os respetivos planetas enquanto estes orbitam o Sol, o ***Scene Graph*** permite acumular matrizes de transformação geométrica. Primeiro, define-se a animação da órbita do planeta em torno da origem através de uma instrução `<rotate time="..." />`. Em seguida, no interior desse grupo, transladam-se os objetos e aplicam-se novas rotações independentes para simular a rotação em torno do próprio eixo e o movimento de eventuais satélites naturais.

O seguinte excerto demonstra o encapsulamento hierárquico utilizado para instanciar o sistema Terra-Lua:

```xml
<group>
    <transform>
        <rotate time="180" x="0" y="1" z="0" />
    </transform>
    <group>
```

```
6           <transform>
7               <translate x="12" y="0" z="0" />
8           </transform>
9           <group>
10              <transform>
11                  <rotate time="6" x="0" y="1" z="0" />
12                  <scale x="0.6" y="0.6" z="0.6" />
13              </transform>
14              <models>
15                  <model file="sphere_1_8_8.3d"> ... </model>
16              </models>
17          </group>
18          <group>
19              <transform>
20                  <rotate time="13.5" x="0" y="1" z="0" />
21                  <translate x="2" y="0" z="0" />
22                  <rotate time="13.5" x="0" y="1" z="0" />
23                  <scale x="0.15" y="0.15" z="0.15" />
24              </transform>
25              <models>
26                  <model file="sphere_1_8_8.3d"> ... </model>
27              </models>
28          </group>
29      </group>
30  </group>
```

Esta arquitetura matricial permite que qualquer corpo adicionado a um subgrupo herde o trajeto espacial completo dos seus nós progenitores, preservando a coerência orbital. Além de translações e rotações, a cena tira partido de curvas animadas baseadas em pontos de controlo — uma funcionalidade complexa exibida por um cometa cujo trajeto é ditado pela ***tag*** `<translate time="135" align="true">` associada a múltiplos pontos cartesianos.

4.4. Texturas Aplicadas na Cena

Para o aspeto visual da cena, foi essencial mapear corretamente texturas de alta e média resolução nas **malhas** geradas. Com base na integração da biblioteca `DevIL` para o carregamento de imagens e cálculo de coordenadas de textura (`UV`) (mecanismos detalhados no Capítulo 3), o **ficheiro `XML`** atribui as seguintes imagens aos respetivos modelos:

- **Esfera envolvente (*Skybox*)**: `8k_stars_milky_way.jpg` (esfera massiva com ***scale*** negativo para projetar os limites do universo visível).
- **Estrela Central (Sol)**: `8k_sun.jpg`.

- **Planetas Rochosos e Luas**: `8k_mercury.jpg`, `8k_venus_surface.jpg`, `8k_earth_daymap.jpg`, `8k_mars.jpg` e `8k_moon.jpg`. Adicionalmente, as luas de Júpiter carregam texturas individualizadas como `4k_io.jpg` e `4k_europa.jpg`.
- **Planetas Gasosos**: `8k_jupiter.jpg`, `8k_saturn.jpg`, `2k_uranus.jpg` e `2k_neptune.jpg`.
- **Cinturas de Asteróides e Cometas**: Fragmentos gerados a partir da **primitiva** extra do cometa utilizam intermitentemente os **ficheiros** `4k_asteroid.jpg` e `4k_comet.jpg`.
- **Anel de Saturno**: Renderizado recorrendo à **primitiva** `torus.3d` (um extra desenvolvido fora do modelo base) em conjunto com a textura transparente `8k_saturn_ring_alpha.png`.

A aplicação sem emendas ou distorções severas destas imagens nos polos das esferas valida a correção dos algoritmos matemáticos geradores do mapa `UV` presentes no módulo `Generator`.

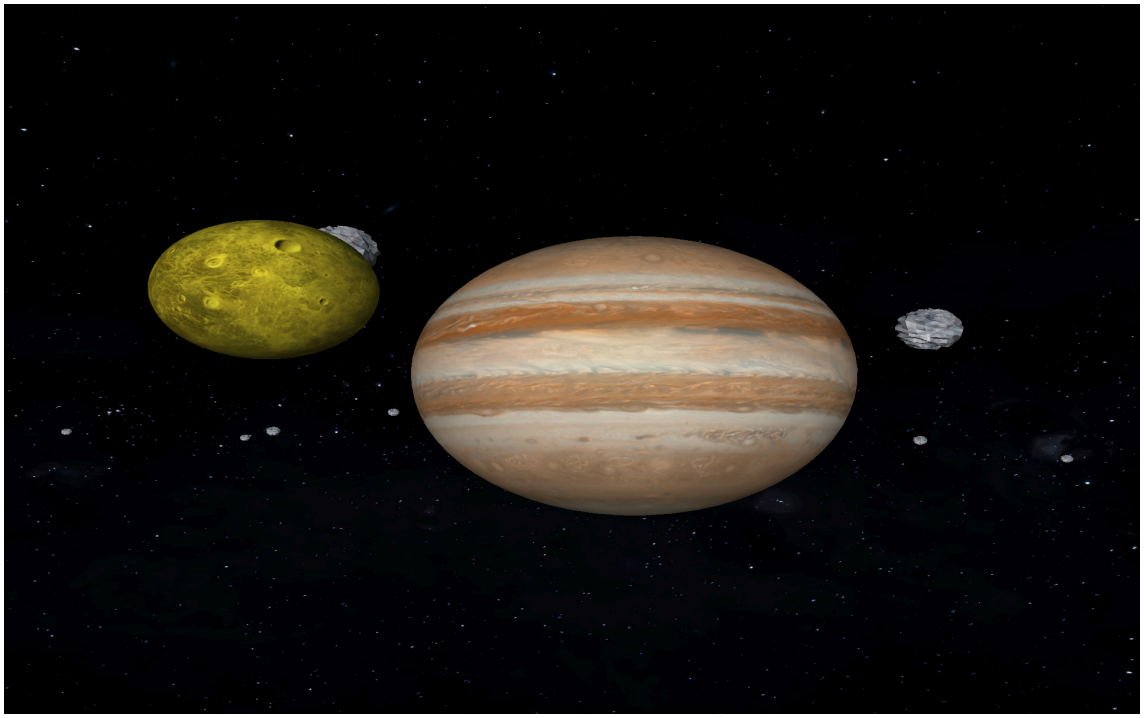


Figura 23: Detalhe das coordenadas UV aplicadas à malha de Júpiter, evidenciando o mapeamento correto da textura gasosa contínua na superfície esférica.

4.5. Configuração de Iluminação e Materiais

Consoante o descrito no Capítulo 2 sobre a integração do motor de luzes fixo (***Fixed--Function Pipeline***), a cena requeria a presença de uma luz central que gerasse sombras e volumes credíveis em todos os corpos esféricos. Assim, declarou-se uma única fonte de luz estática do tipo ***Point Light***, alinhada exatamente com o centro do Sol: `<light type="point" posX="0" posY="0" posZ="0" />`.

Dado que o Sol é o emissor primário na simulação, a sua **malha** foi dotada exclusiva-

mente de um componente emissivo extremo (`<emissive R="255" G="255" B="255" />`), desprovido de reflexos intrínsecos de ambiente ou difusão. Contrastando com o Sol, todos os restantes corpos celestes contêm especificações de resposta fotométrica (Materiais) bastante distintas.

O excerto seguinte documenta o material da Terra, desenhado para absorver uma luz ambiente subtil, refletir uma difusão azulada e, através de um forte componente `<specular>`, acentuar o brilho provocado pela luz da estrela central a colidir com a superfície refletora do globo (oceano):

```xml
<model file="sphere_1_8_8.3d">
    <texture file="../textures/8k_earth_daymap.jpg" />
    <color>
        <diffuse R="100" G="150" B="255" />
        <ambient R="20" G="30" B="50" />
        <specular R="128" G="128" B="128" />
    </color>
</model>
```

Planetas mais áridos, como Marte e Mercúrio, omitem explicitamente as variáveis difusas e especulares acentuadas (`<specular R="0" G="0" B="0" />`), comportando-se visualmente como corpos celestes puramente opacos. A correta distribuição geométrica das normais da fase anterior garante que o lado não iluminado dos planetas permaneça coerentemente na penumbra, suportando-se apenas num baixo espectro de cor ambiente.

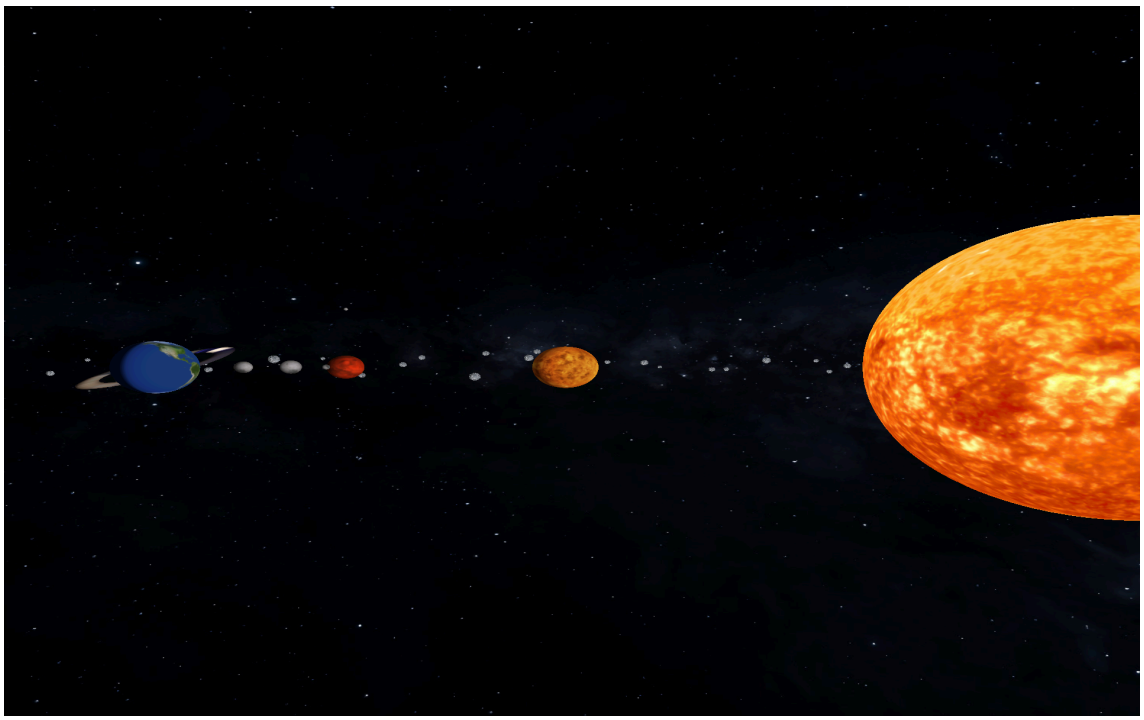


Figura 24: Demonstração do cálculo da luz pontual. A transição clara entre luz difusa e sombra projetada nos planetas evidencia o rigor da normalização vetorial e da configuração da iluminação.

4.6. Resultado Visual Final

O Sistema Solar orquestrado manifesta-se no **ecrã** como um modelo coeso e vibrante de tudo o que foi exigido no contexto da disciplina de Computação Gráfica. A integração de **primitivas** adicionais, com destaque para a inclusão da **malha** do cometa espalhado pelas cinturas de asteróides interiores e exteriores da órbita de Úrano, acrescenta considerável densidade e detalhe ao produto final. Adicionalmente, a translação dinâmica do cometa errante pela cena atesta a viabilidade dos cálculos dos eixos do referencial *Frenet* nas curvas *Catmull-Rom*.

A cena convida a uma exploração ativa. O utilizador pode navegar pelo sistema cósmico recorrendo às lógicas referenciadas no Capítulo 5 (**Conclusão**), nomeadamente através das câmaras **Orbital** (para orbitar visualmente a cena ao redor de pontos específicos de interesse) e ***Free*** (que permite sobrevoar cada planeta à velocidade desejada, operando de forma análoga aos controlos FPS). O volume total de vértices carregados simultaneamente para a VRAM comprova ainda o rigor aplicado à criação das ***buffers*** e dos ponteiros gráficos no ciclo de desenho.

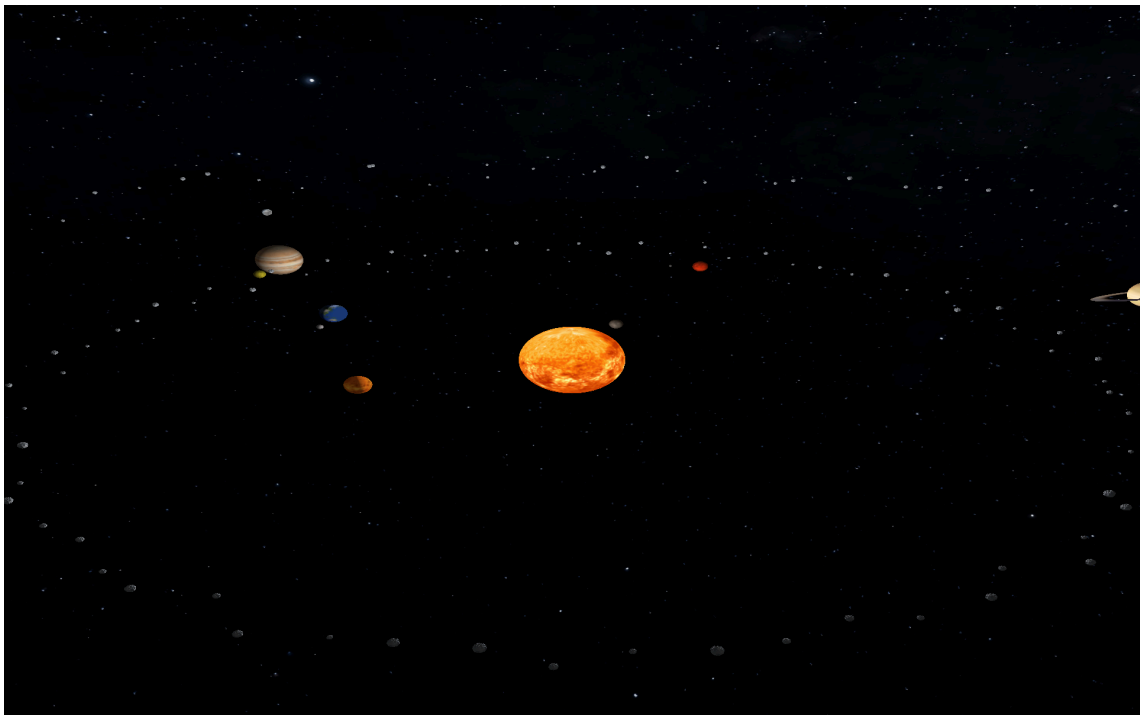


Figura 25: Perspetiva oblíqua do interior do Sistema Solar utilizando a câmara Orbital, exibindo o percurso concêntrico dos planetas rochosos em torno do brilho emissivo da estrela central.

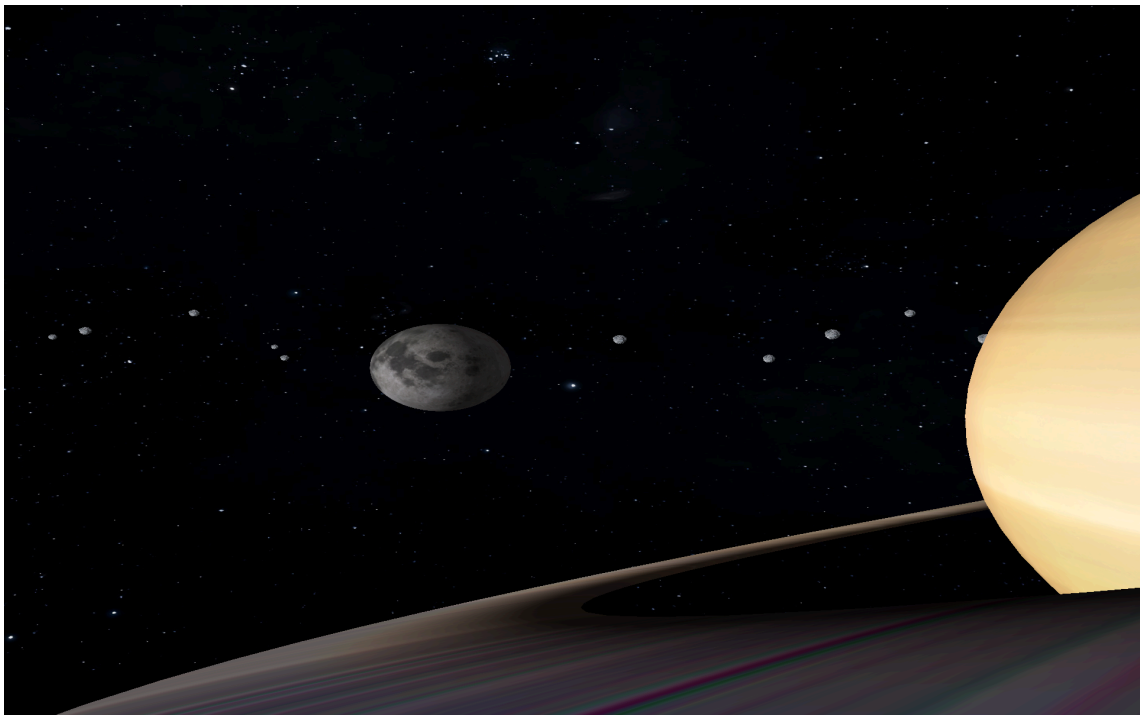


Figura 26: Plano de detalhe obtido via navegação por câmara Livre, com destaque sobre os anéis de Saturno e o satélite natural no plano de fundo.

5. Conclusão

A conclusão desta Fase 4 representa um marco fundamental e o fecho com sucesso do desenvolvimento do nosso Motor 3D. Olhando retrospetivamente para a arquitetura, o sistema provou ser robusto e escalável, suportando a complexidade introduzida pelo *Scene Graph*, pelas transformações em pilha, pelos VBOs e, agora, pelo mapeamento de texturas e pelo modelo de iluminação realista.

O motor atinge um nível de performance satisfatório, lidando bem com cenas complexas, embora a gestão de milhares de polígonos exija sempre uma atenção cuidadosa. Os principais desafios técnicos desta fase final foram, indubitavelmente, o cálculo preciso das normais e coordenadas de textura (UVs) para cada primitiva no *Generator*, bem como garantir que o OpenGL reagisse de forma correta às fontes de luz (em particular, as exigências matemáticas da iluminação tipo *Spotlight* e o posicionamento dinâmico da luz na cena).

De forma a enriquecer o projeto e a demonstrar a flexibilidade da nossa arquitetura, consolidámos um conjunto de "Extras" (desenvolvidos ao longo das várias fases) que consideramos mais-valias significativas. Ao nível da exploração visual, destacamos o sistema duplo de câmaras: a Câmara **Orbital** (ideal para focar e orbitar objetos específicos de forma controlada) e a Câmara **Free** (no estilo FPS, que permite navegar livremente pelo cenário e explorar os modelos a partir de qualquer perspetiva). Expandimos também o leque de primitivas geométricas suportadas com figuras extra, nomeadamente a geração de um **Torus** e de um **Cometa**. Por fim, o nosso *Generator* foi adaptado para exportar os modelos no formato padrão da indústria **Wavefront** (`.obj`), garantindo uma maior interoperabilidade geométrica com outro software 3D.

Como trabalho futuro, identificamos áreas de expansão que poderiam elevar ainda mais a qualidade do motor. A transição do *pipeline* fixo do OpenGL clássico para *Shaders* programáveis modernos (GLSL) permitiria implementar técnicas avançadas como **Phong Shading** calculado por pixel, **Normal Mapping** e sombras dinâmicas em tempo real. O projeto atual deixa-nos com uma fundação extremamente sólida, cumprindo e superando os objetivos propostos para a unidade curricular.

6. Referências Bibliográficas

[1] D. Shreiner, M. Woo, J. Neider, e T. Davis, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3*, 8.º ed. Addison Wesley, 2013.

[2] Khronos Group, «OpenGL Registry - Generating Vertex Normals». [Em linha]. Disponível em: https://www.khronos.org/opengl/wiki/Calculating_a_Surface_Normal

[3] Joey de Vries, «Learn OpenGL - Textures and UV Mapping». [Em linha]. Disponível em: https://learnopengl.com/Getting-started/Textures

[4] Lighthouse3D, «Lighthouse3D - OpenGL Lighting Tutorial». [Em linha]. Disponível em: https://www.lighthouse3d.com/tutorials/glsl-core-tutorial/lighting/